

Το Πέτρινο Σπίτι

Ο Arben κληρονόμησε ένα όμορφο οικόπεδο στους λόφους που βλέπουν το Αργυρόκαστρο. Εμπνευσμένος από τα διάσημα σπίτια της πόλης με τις πέτρινες στέγες, σχεδιάζει να κατασκευάσει ένα δικό του παραδοσιακό σπίτι.

Το οικόπεδο αναπαρίσταται ως ένα τετραγωνικό πλέγμα $N \times N$, όπου οι γραμμές και οι στήλες είναι αριθμημένες από 1 έως N . Το βραχώδες έδαφος έχει μια ιδιαιτερότητα: υπάρχει ακριβώς ένας μεγάλος, ακίνητος βράχος σε κάθε γραμμή και σε κάθε στήλη του πλέγματος.

Για να θέσει τα θεμέλια του σπιτιού του, ο Arben πρέπει να επιλέξει ένα **κενό τετραγωνικό υποπλέγμα** — μια περιοχή μεγέθους $S \times S$ που δεν περιέχει κανέναν βράχο.

Έστω M ο μέγιστος ακέραιος τέτοιος ώστε, για κάθε διάταξη των βράχων που ικανοποιεί τις παραπάνω συνθήκες, να είναι εγγυημένο ότι υπάρχει ένα κενό τετραγωνικό υποπλέγμα μεγέθους $M \times M$. Ο Arben θέλει να βρει *οποιοδήποτε* κενό τετραγωνικό υποπλέγμα μεγέθους τουλάχιστον $M \times M$.

Για να χαρτογραφήσει τη γη, ο Arben ζητάει βοήθεια από τον φίλο του Besa. Ο Besa έχει φέρει μια ειδική συσκευή χαρτογράφησης που ονομάζεται **Gurëmatës** ("μετρητής πέτρας").

Σε μία μονή λειτουργία, ο Besa μπορεί να ρυθμίσει τον Gurëmatës ώστε να σαρώσει ένα ορθογώνιο υποπλέγμα που δίνεται από την πάνω αριστερή γωνία του (x_1, y_1) και την κάτω δεξιά γωνία του (x_2, y_2) . Η συσκευή επιστρέφει τον ακριβή αριθμό των βράχων σε εκείνη την περιοχή. Ο Gurëmatës τροφοδοτείται από μια μικρή μπαταρία και κάθε σάρωση καταναλώνει πολλή ενέργεια, οπότε ο Besa πρέπει να τον χρησιμοποιήσει όσο το δυνατόν λιγότερες φορές.

Γράψτε ένα πρόγραμμα που χρησιμοποιεί τον Gurëmatës για να εντοπίσει ένα κενό τετραγωνικό υποπλέγμα μεγέθους $S \times S$ με $S \geq M$, και αναφέρει τη θέση του.

Λεπτομέρειες υλοποίησης

Πρέπει να υλοποιήσετε την ακόλουθη διαδικασία:

```
std::vector < int > solve(int N);
```

- N : το μέγεθος του πλέγματος.

- Η διαδικασία πρέπει να επιστρέψει ακριβώς τέσσερις ακέραιους $\{x_1, y_1, x_2, y_2\}$: την πάνω αριστερή και την κάτω δεξιά γωνία του κενού τετράγωνου υποπλέγματος που βρήκε. Πρέπει να ισχύει η συνθήκη $x_2 - x_1 = y_2 - y_1 = S - 1$ με $S \geq M$.
- Αυτή η διαδικασία καλείται ακριβώς μία φορά για κάθε περίπτωση ελέγχου.

Η διαδικασία μπορεί να κάνει κλήσεις στην ακόλουθη διαδικασία:

```
int ask(int x1, int y1, int x2, int y2);
```

- x_1, y_1 : η γραμμή και η στήλη της πάνω αριστερής γωνίας του σαρωμένου υποπλέγματος.
- x_2, y_2 : η γραμμή και η στήλη της κάτω δεξιάς γωνίας του σαρωμένου υποπλέγματος.
- Πρέπει να ισχύει ότι $1 \leq x_1 \leq x_2 \leq N$ και $1 \leq y_1 \leq y_2 \leq N$. Διαφορετικά, το πρόγραμμά σας τερματίζεται αμέσως και λαμβάνει 0 πόντους για εκείνη την περίπτωση ελέγχου.
- Η διαδικασία επιστρέφει τον αριθμό των βράχων στο σαρωμένο υποπλέγμα.

Ο βαθμολογητής δεν είναι προσαρμοστικός: οι θέσεις των βράχων είναι καθορισμένες πριν κληθεί η `solve` και δεν εξαρτώνται από τις κλήσεις σας στην `ask`.

Περιορισμοί

- $2 \leq N \leq 500$
- Μπορείτε να κάνετε το πολύ $2.5 \cdot 10^5$ κλήσεις στη `ask`.

Υποπροβλήματα

Υποπρόβλημα	Πόντοι	Επιπλέον περιορισμοί
1	15	$N \leq 40$
2	25	$N \leq 150$
3	60	Χωρίς επιπλέον περιορισμούς.

Βαθμολογία

Αν η `solve` επιστρέψει μια μη έγκυρη περιοχή, κάνει μια μη έγκυρη κλήση στην `ask` ή καλέσει την `ask` περισσότερες από $2.5 \cdot 10^5$ φορές, λαμβάνετε 0 πόντους για αυτή την περίπτωση ελέγχου.

Διαφορετικά, έστω Q ο αριθμός των κλήσεων στην `ask`. Για αυτή την περίπτωση ελέγχου λαμβάνετε τους πόντους του υποπροβλήματος πολλαπλασιασμένους με $\min(1, f(Q))$, όπου

$$f(Q) = \begin{cases} \max(0, -0.05 \cdot Q + 1.55), & \text{if } Q \leq 15 \\ \max(0, -0.19 \cdot \log_{10}(Q) + 1.02), & \text{if } Q > 15 \end{cases}$$

Η βαθμολογία σας για ένα υποπρόβλημα είναι η ελάχιστη βαθμολογία σε όλες τις περιπτώσεις ελέγχου αυτού του υποπροβλήματος. Η τελική βαθμολογία είναι το άθροισμα των βαθμολογιών των υποπροβλημάτων.

Για να σας βοηθήσει να εκτιμήσετε τη βαθμολογία σας, ο παρακάτω πίνακας δείχνει τον προσεγγιστικό πολλαπλασιαστή για μερικές συγκεκριμένες τιμές του Q :

Πλήθος ερωτήσεων (Q)	Πολλαπλασιαστής ($\approx$)
$Q \leq 11$	1.00
12	0.95
15	0.80
50	0.70
100	0.64
1000	0.45
10000	0.26
100000	0.07
$Q \geq 233573$	0.00

Δειγματικός βαθμολογητής (Sample Grader)

Ο δειγματικός βαθμολογητής διαβάζει την είσοδο με την ακόλουθη μορφή:

```
N
c_1 c_2 ... c_N
```

Εδώ, το c_i είναι η στήλη του βράχου στη γραμμή i ($1 \leq c_i \leq N$).

Ο δειγματικός βαθμολογητής εκτυπώνει μία μόνο γραμμή που περιέχει τον πολλαπλασιαστή $\min(1, f(Q))$ που κέρδισαν οι ερωτήσεις σας, ή 0 αν το επιστρεφόμενο υποπλέγμα ή

οποιαδήποτε κλήση της `ask` είναι μη έγκυρη.

Παραδείγματα

Παράδειγμα 1

Είσοδος του δείγματικού βαθμολογητή:

```
4
1 2 3 4
```

Οι βράχοι βρίσκονται στα $(1,1)$, $(2,2)$, $(3,3)$ και $(4,4)$. Το πρόγραμμά σας δεν το γνωρίζει αυτό.

Ο βαθμολογητής καλεί `solve(4)`. Μια πιθανή ακολουθία κλήσεων είναι:

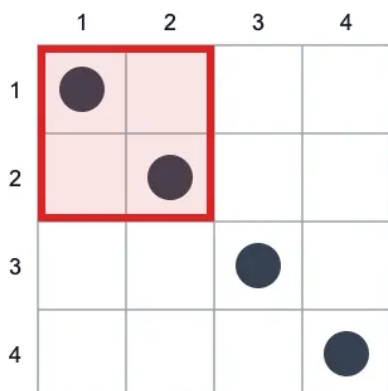
- `ask(1, 1, 2, 2)` επιστρέφει 2.
- `ask(2, 4, 4, 4)` επιστρέφει 1.
- `ask(3, 1, 3, 2)` επιστρέφει 0.

Στη συνέχεια, η `solve` επιστρέφει $\{1, 3, 2, 4\}$.

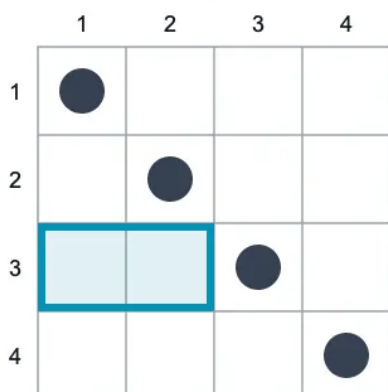
Έξοδος του δείγματος βαθμολογητή:

```
1
```

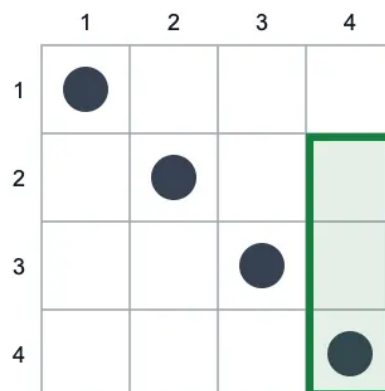
Η πρώτη κλήση σαρώνει το πάνω αριστερό μπλοκ 2×2 και βρίσκει τους βράχους στα $(1,1)$ και $(2,2)$. Η δεύτερη σαρώνει τις γραμμές 2 έως 4 της στήλης 4 και βρίσκει τον βράχο στο $(4,4)$. Η τρίτη σαρώνει τα δύο πρώτα κελιά της γραμμής 3 και δεν βρίσκει βράχους. Η περιοχή που επιστρέφεται καλύπτει τις γραμμές 1 έως 2 και τις στήλες 3 έως 4, η οποία είναι ένα κενό υποπλέγμα 2×2 .



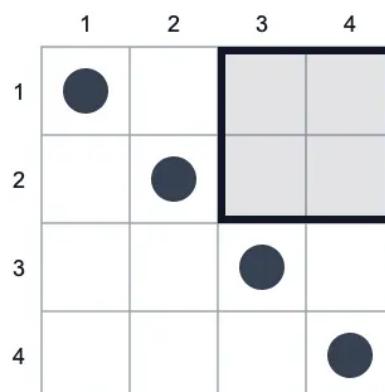
ask(1, 1, 2, 2) returns 2



ask(3, 1, 3, 2) returns 0



ask(2, 4, 4, 4) returns 1



return {1, 3, 2, 4}

Για $N = 4$, το πρόγραμμα χρησιμοποίησε $Q = 3$ κλήσεις, οπότε $f(Q) = 1.4$ και ο πολλαπλασιαστής είναι 1. Άλλες απαντήσεις είναι επίσης αποδεκτές, για παράδειγμα $\{3, 1, 4, 2\}, \{1, 2, 1, 2\}, \{3, 2, 3, 2\}, \{3, 4, 3, 4\}$ ή οποιοδήποτε κενό κελί, αφού $M = 1$ για $N = 4$.